perplexity

4. Recommended Demo Video Script (2 Minutes)

Timestamp	Screen / Visual	Voiceover / Action
0:00 – 0:15	Cursor / Claude Code terminal	"AI agents write 80% of our code today, but they create 'Cognitive Debt'—code in
	our repo nobody actually understands."	
0:15 – 0:40	UnderstandGuard UI	Paste a realistic Git diff containing an API call with an unhandled race
	condition.	
0:40 – 1:10	Reverse Interview Screen	AI fires 3 sharp questions referencing lines in the diff.
1:10 – 1:25	Developer Response	Developer types a surface-level answer for question 2: "It handles errors
	gracefully."	
1:25 – 1:40	Risk Flag + Edge Case	UnderstandGuard flags line 52 as unverified and generates an adversarial test
	proving a silent failure.	
1:40 – 1:55	Export Screen	Displays the generated COGNITIVE_MAP.md and review summary.
1:55 – 2:00	Final Punchline	"UnderstandGuard turns AI-generated code from something you blind-accept into
	something you can explain."	

---

5. Recommended Next Steps for Our Sprint

If you want to move forward with UnderstandGuard:

1. Update Project Scope: Initialize scope.md and prd.md using the Devpost Learn workflow.
2. Select Tech Stack:
   • Frontend: Next.js / Vite + Tailwind (clean, minimal, dark-mode terminal aesthetics).
   • Backend / Engine: Node.js / TypeScript + Gemini API.
   • Diff Parser: Standard diff-parser or Git CLI.
3. Office Hours Check-in: Bring this concept to the Cup of Joe office hours (today at 3:00 PM ET on Discord) for direct feedback
   from Joe.
   about next steps ? .. pls table out the corection if any .....

You're very close to a strong, judge-friendly plan. Your 2-minute script and next steps are solid; I'll just tighten a few things so your video and sprint align tightly with how Devpost judges actually watch and score demos. [1] [2] [3] [4]

1. Demo video script: what to keep vs tweak

What's already good

- Clear problem statement in the first 15 seconds. [3] [1]
- Realistic Git diff with a concrete bug (race condition / unhandled error). [2] [4]
- A natural arc: problem → demo → risk flag → artifact → punchline. [3]
- Strong final line that reframes the value ("blind-accept → explain"). [1]

Targeted corrections / improvements

Timestamp	Your current plan	Suggested correction / refinement	Why it helps for judges
0:00–0:15	Cursor/Claude terminal + voiceover about cognitive debt.	Add **one short on-screen text**: "UnderstandGuard – for teams using AI coding agents." Mention the **hackathon name** once if the rules require it.	Judges often skim; this instantly clarifies *who* it's for and that it's a hackathon submission. [1] [5] [6]
0:15–0:40	Paste a realistic Git diff with race condition.	Show the **file name and a tiny context comment** (e.g., `auth.ts - refresh token flow`). Keep the diff to ≤ **40–50 lines**.	Makes it easier to follow on small screens and keeps the demo tight. [2] [4]
0:40–1:10	Reverse interview: 3 sharp questions.	Ensure each question **mentions line numbers or function names** on screen (e.g., "Line 52 – `refreshToken()`"). Consider showing **Q1 & Q2 fully**, and **summarize Q3** if time is tight.	Judges care more about depth on 1–2 points than rushing through 3. [2] [4]
1:10–1:25	Developer gives a surface-level answer for Q2.	Add a **small on-screen highlight** on the vague phrase ("handles errors gracefully") and a subtle "⚠ Vague" badge.	Visually reinforces the core idea: the tool detects weak understanding, not just "wrong answers." [1]
1:25–1:40	Risk flag + edge-case test showing silent failure.	Explicitly say: "Because Q2 was vague, UnderstandGuard generates this adversarial test for `refreshToken()`." Show **test input → failure → short error**.	Ties the test directly to the interview, which is your unique angle versus generic "AI tester" tools. [3] [4]
1:40–1:55	Export screen: COGNITIVE_MAP.md + review summary.	Show **two concrete artifacts**: - `COGNITIVE_MAP.md` snippet (Mermaid diagram or bullet list). - Verdict badge: "High Cognitive Debt – Review Required."	Judges like tangible outputs they can imagine using in a real repo. [2] [5]
1:55–2:00	Final punchline.	Add **one more sentence** that hints at impact: "For teams, that means fewer surprise bugs and faster, safer reviews."	Connects the clever idea to real-world impact, which many rubrics score explicitly. [7] [8] [3]

General video tips to apply:

- Use a **screencast with clear voiceover**; avoid background music or heavy effects. [2] [1]
- Cut all loading/typing delays; use jump cuts or pre-filled fields. [4] [2]
- Record in **short clips** so you can re-do sections without re-recording everything. [4]
- Upload as **public/unlisted YouTube or Vimeo** and paste the link in your Devpost submission. [9] [10] [11]

2. Next steps: corrections and additions

Your next-step list is directionally correct. Here's a tightened, judge-aligned version.

A. Scope & planning docs

Your plan:

> 1. Update Project Scope: Initialize scope.md and prd.md using the Devpost Learn workflow.

Additions / corrections:

- Create **three files**, not two:
  - `scope.md` – boundaries, MVP, out-of-scope.
  - `prd.md` – user stories, flows, success metrics.
  - `spec.md` – architecture, data model, API contracts, key algorithms.
- In `spec.md`, explicitly describe:
  - How you parse diffs.
  - How you select "critical changes".
  - How you generate questions and score answers.
  - How you generate edge-case tests.

This matches the hackathon's emphasis on planning docs plus working software. [5] [12]

B. Tech stack choices

Your plan:

> 2. Select Tech Stack:
> - Frontend: Next.js / Vite + Tailwind
> - Backend / Engine: Node.js / TypeScript + Gemini API
> - Diff Parser: Standard diff-parser or Git CLI

Corrections / refinements:

Layer	Your choice	Suggested refinement	Reason
Frontend	Next.js / Vite + Tailwind	Use **Next.js + Tailwind** if you're comfortable; otherwise **Vite + React + Tailwind** for speed.	Next.js helps with routing and deployment; Vite is lighter if you want minimal config.
Backend	Node.js / TS + Gemini API	Keep this, but add: **one simple API route** (`/analyze-diff`) that returns: questions, score, risk flags, test snippet.	Makes your demo and codebase cleaner; judges can see a clear API contract. [3]
Diff parsing	diff-parser or Git CLI	Prefer a **library** (e.g., `parse-diff` / `diff` in npm) over shelling out to Git in the demo.	More reliable in hosted environments; easier to show in code walkthroughs.
AI model	Gemini API	Add a **fallback/mock mode** with hardcoded questions/tests for the demo video.	Prevents demo failure if the API is slow or rate-limited during recording. [2] [4]

Also consider:

- A tiny `/demo` **route** pre-loaded with your exact demo diff and answers, so you can just click through during recording. [4]
- A simple **verdict badge component** that switches between "Safe to Review" and "High Cognitive Debt – Review Required". |

C. Office hours / feedback

Your plan:

> 3. Office Hours Check-in: Bring this concept to the Cup of Joe office hours (today at 3:00 PM ET on Discord) for direct feedback from Joe.

This is good. To maximize it:

- Prepare a **1-minute verbal pitch**: problem, who it's for, your 6-step flow, and your demo plan. [3]
- Ask **two specific questions**, for example:
  - "Does this level of 'reverse interview' feel like overkill, or appropriately focused for a PoC?"
  - "Would you prefer we emphasize the cognitive map or the edge-case tests more in the demo?"

That gives you actionable feedback instead of generic "looks good." [2]

D. Missing but important next steps

Add these to your sprint plan:

Step	What to do	Why it matters
4. Define judging story	Write 3–4 bullets mapping your project to typical criteria: Impact, Innovation, Execution, Design, Presentation.	Helps you tune your Devpost description and video script to what judges score. [7] [8] [3]

Step	What to do	Why it matters
5. Prepare demo data	Create: - One sample diff file. - Pre-written “vague” and “good” answers. - One adversarial test case.	Ensures your 2-minute demo is smooth and repeatable. [2] [4]
6. Plan submission text	Draft: - Elevator pitch (1–2 sentences). - “What it does” (3–5 bullets). - “How we built it” (stack + key challenges).	Many judges read this before or instead of watching the full video. [2] [5]
7. Risk plan	Decide: - What if the API is slow? - What if the UI bugs during recording? → Use recorded clips + mock mode.	Prevents last-minute panic and ensures you meet the video deadline. [10] [4]

3. Bottom line

- Your **concept** (UnderstandGuard) is strong and well-aligned with the hackathon theme.
- Your **2-minute script** is almost there; the tweaks above will make it clearer, tighter, and more judge-friendly.
- Your **next steps** are correct in direction; adding planning docs, a mock demo mode, and a focused feedback plan will materially increase your chances.

If you want, next we can:

- Draft a concrete `scope.md` / `prd.md` outline tailored to UnderstandGuard, or
- Turn your 2-minute table into a word-for-word voiceover script you can rehearse.

⁂

1. https://help.devpost.com/article/84-video-making-best-practices
2. https://info.devpost.com/blog/6-tips-for-making-a-hackathon-demo-video
3. https://info.devpost.com/blog/how-to-present-a-successful-hackathon-demo
4. https://webmcp.devpost.com/updates/46161-2-days-left-and-what-judges-actually-look-for
5. https://devpost.com/software/example-template-submission
6. https://pe-hackathon.devpost.com/
7. https://impacthack26.devpost.com/rules
8. https://stempower-her-hacks.devpost.com/?ref_content=default&ref_feature=challenge&ref_medium=portfolio
9. https://help.devpost.com/article/126-know-your-submission-steps
10. https://openai.devpost.com/updates/45402-deadline-tomorrow-last-minute-tips
11. https://revenuecat-shipaton-2026.devpost.com/
12. hackideas.md
13. https://help.devpost.com/article/85-uploading-a-demo-video
14. https://microsoftaiclassroom.devpost.com/updates/28839-how-to-make-your-submission-video-great
15. https://worldslargesthackathon.devpost.com/updates/36832-demo-video-tips
16. https://info.devpost.com/blog/hackathon-judging-tips
17. https://info.devpost.com/blog/understanding-hackathon-submission-and-judging-criteria

18. https://learn-ai-basics.devpost.com/rules